Integrační specifikace — DT Intranet ↔ DT Web

> *Popis datových toků a integračních bodů mezi dt-intranet a dlouhy-web. Verze dokumentu: 2026-03-02*

1. Přehled architektury

```
graph TB
    subgraph "DT Web (port 3001)"
        WEB_PUBLIC["Veřejný web<br/>produkty, reference, blog"]
        WEB_FORMS["Formuláře<br/>poptávka, servis, reklamace"]
        PORTAL["Zákaznický portál<br/>objednávky, faktury, servis"]
        WEBHOOK_HANDLER["Webhook handlery<br/>/api/web/webhooks/*"]
        WEB_DB_READ["Drizzle (read-only)<br/>intranet tabulky"]
        WEB_DB_WRITE["Drizzle (read-write)<br/>web-owned tabulky"]
        EMAIL_SYSTEM["Email systém<br/>Resend + React Email"]
    end

    subgraph "DT Intranet (port 3000)"
        INTRANET_APP["Interní CRM/ERP<br/>pipeline, firmy, nabídky, ...]
        INTRANET_API["REST API<br/>/api/v1/*"]
        INTRANET_WEBHOOKS["Webhook caller<br/>při změně stavu"]
        INTRANET_DB_WRITE["Drizzle (read-write)<br/>všechny tabulky"]
    end

    subgraph "PostgreSQL 16"
        DB[(dt_intranet)]
    end

    %% Čtení z DB
    WEB_DB_READ -->|SELECT| DB
    WEB_DB_WRITE -->|CRUD| DB
    INTRANET_DB_WRITE -->|CRUD| DB

    %% Web → Intranet API
    PORTAL -->|"POST /api/v1/service-requests<br/>POST /api/v1/claims<br/>POST
/api/v1/cases"| INTRANET_API
    WEB_FORMS -.->|"budoucí: forward<br/>leads do intranetu"| INTRANET_API

    %% Intranet → Web webhooky
    INTRANET_WEBHOOKS -->|"POST /webhooks/order-status<br/>POST /webhooks/new-
invoice<br/>POST /webhooks/service-update<br/>POST /webhooks/claim-update"|
WEBHOOK_HANDLER

    %% Webhooky → notifikace + email
    WEBHOOK_HANDLER --> WEB_DB_WRITE
    WEBHOOK_HANDLER --> EMAIL_SYSTEM
```

ASCII verze

```
┌──────────────────────────────────┐      ┌──────────────────────────────────┐
│         DT WEB (:3001)           │      │      DT INTRANET (:3000)         │
│                                  │      │                                  │
│  ┌──────────┐   ┌──────────┐     │      │  ┌──────────┐   ┌──────────┐     │
│  │ Veřejný  │   │  Portál  │     │      │  │ CRM/ERP  │   │ REST API │     │
│  │   web    │   │ zákazník │─────┼──API─▶│  │ moduly   │   │ /api/v1/ │     │
│  └──────────┘   └──────────┘     │      │  └──────────┘   └──────────┘     │
│                      │           │      │                      │           │
│                      │           │      │                      │           │
│  ┌──────────┐   ┌──────────┐     │      │  ┌──────────┐   ┌──────────┐     │
│  │ Webhook  │◀──│ Notif +  │     │      │  │ Webhook  │   │ DB write │     │
│  │ handlery │   │  Email   │     │  ◀───┼──│ caller   │   │ (CRUD)   │     │
│  └──────────┘   └──────────┘     │  WH  │  └──────────┘   └──────────┘     │
│        │                         │      │                      │           │
│        │                         │      │                      │           │
│  ┌──────────────────────────┐    │      │  ┌──────────────────────────┐    │
│  │  Drizzle READ-ONLY       │    │      │  │  Drizzle READ-WRITE      │    │
│  │  (intranet tabulky)      │────┼──┐   │  │  (všechny tabulky)       │────┤
│  └──────────────────────────┘    │  │   │  └──────────────────────────┘    │
│                                  │  │   │                                  │
│  ┌──────────────────────────┐    │  │   │                                  │
│  │  Drizzle READ-WRITE      │────┼──┤   │                                  │
│  │  (web-owned tabulky)     │    │  │   │                                  │
│  └──────────────────────────┘    │  │   │                                  │
└──────────────────────────────────┘  │   └──────────────────────────────────┘
                                      │
                                      │
                                  ┌───┴──────────┐
                                  │ PostgreSQL 16│
                                  │ dt_intranet  │
                                  └──────────────┘
```

---

2. Sdílená databáze

Architektura

Oba systémy používají **jednu PostgreSQL 16 databázi** (`dt_intranet`). Web má omezenou connection pool (`max: 5`).

Tabulky které web čte z intranetu

Tabulka	Klíčová pole	Jak web filtruje	Účel v portálu
`orders`	id, companyId, orderNumber, title, amount, status, timestamps	`WHERE companyId = session.user.companyId`	Seznam objednávek, detail, timeline
`invoices`	id, companyId, invoiceNumber, amount, status, dueDate, paidAt	`WHERE companyId = session.user.companyId`	Seznam faktur, detail, PDF
`service_requests`	id, companyId, title, priority, status, scheduledDate	`WHERE companyId = session.user.companyId`	Seznam servisů, detail
`claims`	id, companyId, title, status, slaDeadline	`WHERE companyId = session.user.companyId`	Seznam reklamací, detail

`companies`	id, name, ico, dic, addresses, contacts	`WHERE id = session.user.companyId`	Firemní info v dashboardu

Pravidlo: Web = READ-ONLY

Web **nikdy** nezapisuje přímo do intranetových tabulek. Pro vytváření entit (servis, reklamace, poptávky) volá Intranet REST API.

Drizzle schema kopie

Web definuje read-only Drizzle schémata v `src/lib/db/schema/intranet/` :

```
// src/lib/db/schema/intranet/orders.ts
import { pgTable, uuid, varchar, integer, timestamp, date } from "drizzle-orm/pg-core"

export const intranetOrders = pgTable("orders", {
  id: uuid("id").primaryKey(),
  companyId: uuid("company_id"),
  orderNumber: varchar("order_number", { length: 50 }),
  title: varchar("title", { length: 500 }),
  type: varchar("type", { length: 20 }),
  amount: integer("amount"),
  currency: varchar("currency", { length: 3 }),
  status: varchar("status", { length: 20 }),
  orderedAt: timestamp("ordered_at", { withTimezone: true }),
  confirmedAt: timestamp("confirmed_at", { withTimezone: true }),
  shippedAt: timestamp("shipped_at", { withTimezone: true }),
  deliveredAt: timestamp("delivered_at", { withTimezone: true }),
  deliveryDate: date("delivery_date"),
  note: varchar("note"),
  createdAt: timestamp("created_at", { withTimezone: true }),
  updatedAt: timestamp("updated_at", { withTimezone: true }),
  deletedAt: timestamp("deleted_at", { withTimezone: true }),
})
```

Importy:

```
import {
  intranetOrders,
  intranetInvoices,
  intranetCompanies,
  intranetServiceRequests,
  intranetClaims,
} from "@/lib/db/schema/intranet"
```

---

3. Intranet API → Web (web volá intranet)

HTTP client

Soubor: `src/lib/intranet-client.ts`

```
const INTRANET_URL = process.env.INTRANET_API_URL || "http://localhost:3000"
const INTRANET_API_KEY = process.env.INTRANET_API_KEY

async function intranetFetch(path: string, options: RequestInit) {
  return fetch(`${INTRANET_URL}${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      "X-API-Key": INTRANET_API_KEY,
      ...options.headers,
    },
  })
}
```

Endpointy které web volá

`POST /api/v1/cases` — nová poptávka z webu

```
await createBusinessCase({
  org_id: string,       // default org
  company_id: string,   // z portal session
  title: string,
  description: string,
  source: "web",        // vždy "web"
  status: "lead",       // default
})
```

Kdy se volá: Formuláře z portálu, budoucí forward web_leads.

`POST /api/v1/service-requests` — nový servisní požadavek

```
await createServiceRequest({
  org_id: string,
  company_id: string,   // z portal session
  title: string,
  description: string,
  priority: "normal" | "high" | "critical",
  scheduled_date?: string,
})
```

Kdy se volá: Portál → `/portal/servis/novy` → `POST /api/web/portal/service-requests`.

`POST /api/v1/claims` — nová reklamace

```
await createClaim({
  org_id: string,
  company_id: string,   // z portal session
```

```
  title: string,
  description: string,
})
```

Kdy se volá: Portál → `/portal/reklamace/nova` → `POST /api/web/portal/claims`.

Autentizace

Všechny requesty obsahují header:

```
X-API-Key: ${INTRANET_API_KEY}
```

Mock mód

Pokud `PORTAL_MOCK=true`:

- Všechny intranet queries vrací mock data
- Nezávisí na běžícím intranetu
- Mock data: 3 objednávky, 3 faktury, 2 servisní požadavky, 1 reklamace, sample company

---

4. Webhooky — Intranet → Web (intranet notifikuje web)

Intranet volá web webhook endpointy při změně stavu entit.

Společná autentizace

Každý webhook vyžaduje:

```
X-API-Key: ${WEBHOOK_SECRET}
```

Společný handler flow

1. Validace `X-API-Key` headeru
2. Parse JSON body
3. Zod validace against schema
4. Lookup aktivních portal_users pro danou firmu (`companyId`)
5. INSERT `portal_notifications` pro každého uživatele
6. Odeslání emailu uživatelům s povolenými notifikacemi
7. Return `{ success: true }`

`POST /api/web/webhooks/order-status`

Zod schema:

```
const orderStatusWebhookSchema = z.object({
  orderId: z.string().uuid(),
  orderNumber: z.string(),
  companyId: z.string().uuid(),
  status: z.enum(["pending", "confirmed", "shipped", "completed"]),
  description: z.string().optional(),
  updatedAt: z.string().datetime(),
})
```

Akce:

- Notification type: `order_status`
- Link: `/portal/objednavky/${orderId}`
- Email template: `OrderStatusNotification`

Status labels:

Status	Label
pending	Čekající
confirmed	Potvrzeno
shipped	Odesláno
completed	Dokončeno

`POST /api/web/webhooks/new-invoice`

Zod schema:

```
const newInvoiceWebhookSchema = z.object({
  invoiceId: z.string().uuid(),
  invoiceNumber: z.string(),
  companyId: z.string().uuid(),
  orderId: z.string().uuid().optional(),
  amount: z.number(),              // v haléřích
  dueDate: z.string().datetime(),
  createdAt: z.string().datetime(),
})
```

Akce:

- Notification type: `new_invoice`
- Link: `/portal/faktury/${invoiceId}`
- Email template: `InvoiceNotification`

`POST /api/web/webhooks/service-update`

Zod schema:

```
const serviceUpdateWebhookSchema = z.object({
  serviceRequestId: z.string().uuid(),
  requestNumber: z.string(),
  companyId: z.string().uuid(),
  status: z.enum(["new", "in_progress", "waiting_parts", "completed", "cancelled"]),
  description: z.string().optional(),
  updatedAt: z.string().datetime(),
})
```

Akce:

- Notification type: `service_update`

- Link: `/portal/servis/${serviceRequestId}`
- Email template: `ServiceUpdateNotification`

Status labels:

Status	Label
new	Nový
in_progress	V realizaci
waiting_parts	Čeká na díly
completed	Dokončeno
cancelled	Zrušeno

`POST /api/web/webhooks/claim-update`

Zod schema:

```
const claimUpdateWebhookSchema = z.object({
  claimId: z.string().uuid(),
  claimNumber: z.string(),
  companyId: z.string().uuid(),
  status: z.enum(["new", "investigating", "approved", "rejected", "resolved"]),
  description: z.string().optional(),
  updatedAt: z.string().datetime(),
})
```

Akce:

- Notification type: `claim_update`
- Link: `/portal/reklamace/${claimId}`
- Email template: `ClaimUpdateNotification`

Status labels:

Status	Label
new	Nová
investigating	V šetření
approved	Schváleno
rejected	Zamítnuto
resolved	Vyřešeno

---

5. Datové toky — přehledová tabulka

Intranet → Web (čtení z DB)

Data	Tabulka	Filtr	Použití v portálu
Objednávky	`orders`	`companyId`	Dashboard KPI, list, detail, timeline
Faktury	`invoices`	`companyId`	Dashboard KPI, list, detail, PDF
Servisní požadavky	`service_requests`	`companyId`	List, detail
Reklamace	`claims`	`companyId`	List, detail
Firemní info	`companies`	`id`	Dashboard, profil

Intranet → Web (push přes webhooky)

Událost	Webhook	Akce na webu
Změna stavu objednávky	`order-status`	Notifikace + email
Nová faktura	`new-invoice`	Notifikace + email
Update servisu	`service-update`	Notifikace + email
Update reklamace	`claim-update`	Notifikace + email

Web → Intranet (zápis přes API)

Akce	Web endpoint	Intranet endpoint	Trigger
Nový servisní požadavek	`POST /api/web/portal/service-requests`	`POST /api/v1/service-requests`	Portálový formulář
Nová reklamace	`POST /api/web/portal/claims`	`POST /api/v1/claims`	Portálový formulář
Nová poptávka (business case)	`POST /api/web/portal/*` (budoucí)	`POST /api/v1/cases`	Portálový formulář

Web-only data (nikdy nejde do intranetu)

Data	Tabulka	Popis
Portáloví uživatelé	`portal_users`	Auth, profily, invite flow
Portálové notifikace	`portal_notifications`	Bell icon, čtení stavu
Web leads	`web_leads`	Veřejné formuláře (poptávka, reklamace, servis, kontakt)
Produkty	`products`, `categories`, `product_*`	CMS obsah
Články	`articles`	Blog

Reference	`references`	Case studies
Downloads	`downloads`	Dokumenty ke stažení
CMS obsah	`web_page_content`	Editovatelný obsah stránek

---

6. Bezpečnost

CompanyId izolace

Zákazník v portálu vidí **pouze data své firmy**:

```
// Každý portálový query:
const session = await getPortalSession()
const orders = await db
  .select()
  .from(intranetOrders)
  .where(and(
    eq(intranetOrders.companyId, session.user.companyId),
    isNull(intranetOrders.deletedAt)
  ))
```

API key autentizace

Směr	Header	Env proměnná
Web → Intranet API	`X-API-Key: ${INTRANET_API_KEY}`	`INTRANET_API_KEY` (na webu)
Intranet → Web webhooky	`X-API-Key: ${WEBHOOK_SECRET}`	`WEBHOOK_SECRET` (na obou stranách)
Import veřejných zakázek	`X-API-Key: ${API_IMPORT_KEY}`	`API_IMPORT_KEY` (na intranetu)

Žádný přímý zápis do intranetových tabulek

Web nikdy nepoužívá `INSERT`, `UPDATE`, `DELETE` na intranetových tabulkách. Veškeré zápisy jdou přes Intranet REST API, kde se:

- Validuje vstup (Zod)
- Kontroluje oprávnění (RBAC)
- Loguje akce (audit_log)
- Odesílají notifikace

Rate limiting

Veřejné formuláře: **10 requestů / IP / minutu**.

Webhook validace

Každý webhook handler:

1. Ověří `X-API-Key` header
2. Validuje body přes Zod schema
3. Při nevalidním requestu vrací 401 nebo 400

---

7. Env proměnné

DT Web (dlouhy-web)

```
# Připojení k DB (sdílená s intranetem)
DATABASE_URL=postgresql://dt:password@localhost:5432/dt_intranet

# Intranet API (web → intranet)
INTRANET_API_URL=http://localhost:3000
INTRANET_API_KEY=<shared-api-key>

# Webhooky (intranet → web)
WEBHOOK_SECRET=<shared-webhook-secret>

# Auth
AUTH_SECRET=<web-auth-secret>
AUTH_URL=https://dlouhy-technology.cz

# Email
RESEND_API_KEY=re_xxx

# ISR revalidation
REVALIDATE_TOKEN=<revalidate-token>

# Payload CMS
PAYLOAD_SECRET=<payload-secret>

# Cloudflare R2
R2_ACCOUNT_ID=xxx
R2_ACCESS_KEY_ID=xxx
R2_SECRET_ACCESS_KEY=xxx
R2_BUCKET_NAME=dt-web
R2_PUBLIC_URL=https://pub-xxx.r2.dev

# Mock mód (portál bez intranetu)
PORTAL_MOCK=false
```

DT Intranet (dt-intranet)

```
# DB
DATABASE_URL=postgresql://dt:password@localhost:5432/dt_intranet

# Auth
NEXTAUTH_URL=https://intranet.dlouhytechnology.com
NEXTAUTH_SECRET=<intranet-auth-secret>

# API klíč pro příchozí requesty z webu
# (validuje se proti X-API-Key headeru)
```

```
# Webhook secret pro odchozí webhooky na web
WEBHOOK_SECRET=<shared-webhook-secret>

# Web URL pro odesílání webhooků
WEB_WEBHOOK_URL=https://dlouhy-technology.cz/api/web/webhooks

# Cron
CRON_SECRET=<cron-secret>

# Email
SMTP_HOST=<smtp-server>
SMTP_PORT=587
SMTP_USER=<email>
SMTP_PASS=<password>
EMAIL_FROM=DT Intranet <noreply@dlouhytechnology.com>
```

Sdílené secrets (musí být stejné na obou stranách)

Secret	Web env	Intranet env	Účel
API key	`INTRANET_API_KEY`	(validace v API handleru)	Web volá intranet API
Webhook secret	`WEBHOOK_SECRET`	`WEBHOOK_SECRET`	Intranet posílá webhooky na web
DB connection	`DATABASE_URL`	`DATABASE_URL`	Sdílená PostgreSQL instance